PySpark Out of Memory (OOM) — Practical Cheatsheet

🧠 PySpark OOM isn't just "too much data." It's often about uneven data distribution and heavy shuffles.

Common Triggers

- Skewed keys during aggregations or joins
- Wide shuffles from groupBy, join, distinct, orderBy
- Too few partitions for large datasets
- Insufficient executor or driver memory
- Exploding rows from `explode` or `crossJoin`

```
# Risky pattern if customer_id is skewed
from pyspark.sql import functions as F

df = spark.read.csv("/mnt/data/transactions.csv", header=True, inferSchema=True)
result = df.groupBy("customer_id").agg(F.sum("purchase_amount").alias("total"))
result.show()
```

Fix 1: Repartition to Distribute Load

Distribute skewed keys across more tasks to avoid single-executor overload.

```
from pyspark.sql import functions as F
```

```
# Hash partition by customer_id for better balance
df = df.repartition(200, "customer_id")

result = df.groupBy("customer_id").agg(F.sum("purchase_amount").alias("tota
l_purchase"))
```

Tips:

- Start around 2–3x total cores in the cluster, then tune
- Use the skewed column as the partition column when possible

---

Fix 2: Aggregate Earlier and Reduce Shuffles

Push reductions close to the source. Consider RDD ops if needed.

```
from pyspark.sql import functions as F

# DataFrame approach with pre-aggregation
partial = df.groupBy("customer_id").agg(F.sum("purchase_amount").alias("par
tial_sum"))

# Or RDD for tight control
rdd = df.select("customer_id", "purchase_amount").rdd.map(
    lambda r: (r["customer_id"], r["purchase_amount"]))
agg_rdd = rdd.reduceByKey(lambda x, y: x + y)
result = agg_rdd.toDF(["customer_id", "total_purchase"])
```

Tips:

- Avoid unnecessary columns before shuffles: select only needed fields
- Cache strategically if reused, but unpersist when finished

Fix 3: Tune Spark Memory and Shuffle Behavior

Increase memory and optimize shuffle handling.

```
spark-submit \
  --executor-memory 8G \
  --driver-memory 4G \
  --conf spark.memory.fraction=0.8 \
  --conf spark.sql.shuffle.partitions=400 \
  --conf spark.reducer.maxSizeInFlight=96m \
  --conf spark.shuffle.file.buffer=64k \
  my_app.py
```

Tips:

- Increase `spark.sql.shuffle.partitions` for large shuffles
- Ensure enough executors and cores to use partitions effectively

---

Fix 4: Handle Skew with Salting

Break up heavy keys into multiple buckets, then aggregate back.

```
from pyspark.sql import functions as F

k = 10  # number of salt buckets; tune based on skew severity
salted = df.withColumn("salt", (F.rand() * k).cast("int"))
salted = salted.withColumn(
    "salted_id", F.concat_ws("_", F.col("customer_id"), F.col("salt"))
)

salted_agg = salted.groupBy("salted_id").agg(
    F.sum("purchase_amount").alias("partial_sum")
)

final = salted_agg.withColumn(
    "customer_id", F.split(F.col("salted_id"), "_")[0]
).groupBy("customer_id").agg(
    F.sum("partial_sum").alias("total_purchase")
)
```

Tips:

- Apply salting only to top skewed keys to avoid overhead
- Identify heavy hitters via frequency checks

Fix 5: Join Strategies for Skew

- Broadcast small tables to avoid big shuffles:

```
from pyspark.sql.functions import broadcast

result = large_df.join(broadcast(small_df), "customer_id", "inner")
```

- For skewed joins, combine salting on the large side with broadcast on the small side

---

Diagnosis Checklist

- Check partition counts: `df.rdd.getNumPartitions()`
- Inspect skew: `df.groupBy("customer_id").count().orderBy(F.desc("count")).show()`
- Review shuffle metrics: Spark UI → SQL tab → Stage DAGs
- Watch for spill and OOM in executor logs

Balanced vs Skewed

- Balanced: even partition sizes, consistent task durations
- Skewed: a few huge partitions, long-running or failing tasks

---

Closing tip: Mix strategies — repartitioning, early aggregation, memory tuning, salting, and broadcast joins. Small, targeted changes often eliminate OOM without oversizing the cluster.

Prepared by **Sai Kiran Peta**